Actualización documental de C extended

Revisión del 15 de octubre de 2026 vinculada al Entregable 4. Este documento contiene textos y tablas para incorporar como actualización posterior a los entregables 1, 2 y 3. Conserva el significado de sus fechas y resultados históricos; los archivos originales permanecen intactos.

Los encabezados de destino se tomaron de los documentos reales. La lista completa de cambios y su evidencia está en Cambios_lexicos_y_ajustes_entregables.md. Las actualizaciones de especificación no deben confundirse con nuevas funciones del motor: varias capacidades ya existían antes de esta revisión.

Actualización del Entregable 1

Documento de destino: Especificación_léxica_y_diseño_conceptual.docx. Añadir una nota de revisión posterior y usar los bloques siguientes en una copia revisada. Mantener la elección del subconjunto propio basado en C#, la bitácora y la fecha histórica de la entrega original.

Listado de Palabras Reservadas

Texto de actualización: C-extended conserva las trece palabras iniciales y utiliza treinta palabras reservadas. La comparación es ordinal y sensible a mayúsculas; solo el identificador completo se compara con la lista. Las ampliaciones ya estaban presentes en el núcleo suministrado para revisión, por lo que se documentan como continuidad y no como treinta nuevas implementaciones.

Conjunto	Palabras
Inicial, 13 palabras	namespace, class, enum, func, var, nil, if, else, for, while, return, break, continue
Ampliación ya presente, 17 palabras	int, float, double, char, bool, string, void, static, true, false, switch, case, default, public, private, using, new

Matriz de Tokens y Expresiones Regulares

Insertar estas definiciones auxiliares antes de sustituir la matriz. Las expresiones son notación formal para documentar el lenguaje; el programa reconoce manualmente los caracteres y no ejecuta estas expresiones con Regex de .NET.

Símbolo	Definición exacta
Σ	Conjunto de unidades UTF-16 (char de .NET) que recibe Analizar(string). El lector de la interfaz valida la codificación y rechaza NUL antes de cargar el editor; esa validación de archivo es una etapa diferente.
L	Unidad UTF-16 para la que char.IsLetter devuelve true. El guion bajo se define aparte. Se admiten letras como ñ y á; no se afirma soporte de todos los puntos de código Unicode ni de todas las secuencias combinantes.
D	Conjunto de los diez dígitos ASCII: {0,1,2,3,4,5,6,7,8,9}. Lo reconoce AutomataAFD.EsDigito.
G	El carácter guion bajo: _.

Símbolo	Definición exacta
E	Caracteres que pueden seguir a una barra invertida en un escape: n, t, r, 0, barra invertida, comilla doble y comilla simple. Son siete caracteres; no son siete saltos o valores ya decodificados.
C_s	Σ excepto comilla doble, barra invertida, CR y LF.
C_c	Σ excepto comilla simple, barra invertida, CR y LF.
C_l	Σ excepto CR y LF.
A y B	A = Σ excepto asterisco. B = Σ excepto asterisco y diagonal /.
Notación	| significa unión, · concatenación y * cerradura de Kleene. Los paréntesis agrupan; las comillas dobles tipográficas « » delimitan caracteres o cadenas literales. X+ es solo abreviatura de X·X*: en la matriz se usa su expansión.

Categoría	Expresión formal	Tres lexemas válidos
TK_RESERVADA	«namespace» | «class» | «enum» | «func» | «var» | «nil» | «if» | «else» | «for» | «while» | «return» | «break» | «continue» | «int» | «float» | «double» | «char» | «bool» | «string» | «void» | «static» | «true» | «false» | «switch» | «case» | «default» | «public» | «private» | «using» | «new»	var ; enum ; func
TK_IDENTIFICADOR	(L | G)·(L | D | G)*, siempre que el lexema completo no pertenezca a las reservadas.	totalVentas ; _i ; niño2
TK_NUM_ENTERO	D·D*	0 ; 15 ; 2026
TK_NUM_DECIMAL	D·D*·«.»·D·D*	3.1416 ; 0.5 ; 12.0
TK_CADENA	«"»·(C_s | «\»·E)*·«"»	"Activo" ; "A\tB" ; "Hola, \"C\""
TK_CARACTER	«'»·(C_c | «\»·E)·«'»	'a' ; '9' ; '\n'
TK_OP_ARITMETICO	«+» | «-» | «*» | «/» | «%» | «++» | «--»	+ ; * ; ++
TK_OP_RELACIONAL	«==» | «!=» | «<» | «>» | «<=» | «>=»	== ; <= ; !=
TK_OP_LOGICO	«&&» | «||» | «!»	&& ; || ; !
TK_OP_ASIGNACION	«=» | «+=» | «-=» | «*=» | «/=» | «%=»	= ; += ; %=
TK_DELIMITADOR	«(» | «)» | «{» | «}» | «[» | «]» | «;» | «,» | «.» | «:»	«{», «;», «:»
TK_COMENTARIO_LINEA	«//»·C_l*	// uno ; // dos ; // @
TK_COMENTARIO_BLOQUE	«/*»·(A | «*»·«*»*·B)*·«*»·«*»*·«/»	/**/ ; /*nota*/ ; /* * */

Los delimitadores de la notación no forman parte del lexema: por ejemplo, «//» representa las dos diagonales. En TK_CARACTER, las comillas simples de apertura y cierre sí pertenecen al lexema. Los ejemplos con barra invertida muestran el código fuente, no una cadena ya desescapada.

La expresión de comentario de bloque obliga a que el primer cierre */ termine el token. A no contiene asterisco y B no contiene asterisco ni diagonal, por lo que el cuerpo no puede absorber un cierre y elegir uno posterior. Los comentarios no son anidados y pueden contener saltos de línea.

Prioridades: los comentarios se prueban antes de la división y los operadores de dos caracteres antes de los de uno. Las cadenas, caracteres y comentarios conservan su lexema original. Un prefijo de una reservada no convierte otro identificador en reservado.

Convenciones y límites que deben acompañar la matriz

Línea y columna comienzan en 1. CRLF cuenta como un único salto; LF y CR aislados también incrementan la línea y restablecen la columna a 1. Una tabulación incrementa la columna en una unidad del recorrido, aunque el editor la dibuje con otra anchura. TotalLineas incluye líneas vacías y la línea vacía final cuando existe un salto al final del texto.

Los números no incluyen signo, exponente ni prefijos de base. -12 produce el operador - y el entero 12. .5 produce el delimitador . y el entero 5: no es un decimal admitido por esta gramática. 1. es un error; 1.2 es válido. Una continuación numérica inválida se consume como una secuencia completa antes de reanudar en el separador.

Un carácter literal contiene exactamente una unidad UTF-16 o un escape permitido. Un símbolo que ocupa dos unidades UTF-16 no se acepta como un único carácter literal. Las letras de identificadores siguen el predicado indicado para L, no una promesa general de aceptar cualquier carácter Unicode.

Reglas para la Detección de Errores Léxicos

Regla	Ejemplo de entrada	Secuencia y recuperación
Identificador con inicio numérico	12variable; final	Error por 12variable. Se conserva ; para reconocerlo después y continúa hasta final.
Decimal con continuación inválida	45.12.3; final o 1.2abc; final	Un error por el lexema completo; se consumen letras, dígitos, guion bajo y puntos de la continuación inválida hasta el separador.
Falta dígito tras el primer punto	12.abc; final o 1..2abc; final	Error por 12.abc o 1..2abc completos. No se agrega abc como identificador.
Cadena sin cerrar	Comilla doble seguida de contenido y fin de línea o EOF	Se reporta desde la comilla inicial hasta antes del salto; el salto queda para el recorrido principal. En EOF termina el análisis.
Escape inválido en cadena	"A\q"; final	Si existe cierre, el error incluye toda la cadena con sus comillas. Un escape inválido no genera otro token independiente.
Cantidad inválida en carácter literal	''; final o 'ab'; final	Error por el literal cerrado completo; debe contener una unidad o un escape válido.
Escape inválido en carácter	'\q'; final	Error por el literal completo; continúa después de su cierre.
Carácter literal sin cerrar	Comilla simple abierta hasta salto o EOF	Error hasta antes del salto o hasta EOF; puede continuar en la línea siguiente si existe.
Comentario de bloque sin cerrar	/* comentario hasta EOF	Consume el resto del archivo y genera un error. Un @ dentro de ese comentario no es otro error.

Regla	Ejemplo de entrada	Secuencia y recuperación
Carácter no permitido	@ var final = 1; o #	Consume la unidad inválida y continúa; la posición indica su inicio.

Los diagnósticos se refieren al comienzo de la secuencia errónea. Si una cadena o un carácter no tiene cierre, se prioriza el diagnóstico de falta de cierre. En caracteres cerrados, el escape inválido se informa antes que la cantidad incorrecta de unidades.

Estructura Preliminar de la Tabla de Símbolos

Texto de actualización: la estructura final conserva los cuatro atributos originales. Simbolo está definido en TablaDeSimbolos.cs; sus propiedades son públicas para lectura y tienen asignación privada. La tabla usa Dictionary<string, Simbolo> con StringComparer.Ordinal y una lista para conservar el orden de primera aparición.

Atributo	Tipo	Significado
NombreIdentificador	string	Lexema exacto del identificador.
TipoToken	string	TK_IDENTIFICADOR; categoría léxica, no tipo semántico de variable.
LineaPrimeraAparicion	int	Primera línea, desde 1.
ColumnaInicio	int	Primera columna, desde 1.

Mecanismo de Control de Duplicados

AgregarSimbolo registra únicamente identificadores que todavía no están en el diccionario. Repetir el mismo identificador conserva la primera ubicación. Nombres que difieren en mayúsculas son distintos. ObtenerSimbolos devuelve una lista nueva en orden de registro. Estas garantías ya existían en el motor revisado; aquí se documenta su forma real.

Actualización del Entregable 2

Documento de destino: SEGUNDO ENTREGABLE - ANALIZADOR LEXICO.dotx. Actualizar Sección 1 - Diagrama del AFD, Sección 2 - Tabla de Transiciones y Sección 3 - Arquitectura UML. La nomenclatura siguiente sustituye las etiquetas contradictorias de las versiones previas; los nombres Q son estados conceptuales de los reconocedores, no nuevos campos ni una tabla ejecutada por el código.

Sección 1 Diagrama del AFD

El dibujo original usa S6 para aceptación de cadena y S7 para barra, mientras sus tablas llaman S6 a barra y S7 al comentario de línea. Sustituir esas etiquetas por la nomenclatura única de las tablas siguientes. El reconocimiento real se distribuye en métodos manuales de AnalizadorLexico; AutomataAFD contiene predicados, reservadas y escapes, no el bucle principal del autómata.

Estado o familia	Función y aceptación
Q0	Despachador inicial del siguiente lexema. Ignora blancos, selecciona reconocedor y termina en EOF.
QI	Identificador/reservada: finaliza al encontrar un carácter que no sea L, D o G.
QN, QP, QD	Parte entera; punto que exige un dígito; parte decimal. QN y QD pueden aceptar al llegar a un separador o EOF.
QNE	Recuperación numérica. Consume R = L ∪ D ∪ G ∪ {punto}; conserva el separador posterior. El motivo depende de la rama de entrada.
QS y QSE	Cadena normal y carácter posterior a barra invertida. Se conserva una marca de escape inválido.
QCk y QCEk	Carácter literal y su escape, con k = 0, 1 o más de 1 unidades. La aceptación por cierre exige k=1 y escapes válidos.
QL	Comentario de línea: acepta antes de CR/LF o en EOF.
QB y QB*	Comentario de bloque y posible cierre después de uno o más asteriscos.
QO	Operador/delimitador: prueba primero pares válidos, después símbolos simples.
A y ERR	Salidas de aceptación y diagnóstico; el control vuelve a Q0 si queda entrada. No son errores semánticos.

Instrucción gráfica: redibujar un despachador Q0 y subautómatas separados para identificadores, números, cadenas, caracteres, comentarios y operadores. Marcar el inicio, las salidas de aceptación y error y todas las transiciones de las tablas. La revisión de esta etapa entrega las tablas y la descripción exacta para sustituir el dibujo; no se presenta una imagen nueva como si ya hubiera sido incorporada al DOTX histórico. La revisión gráfica del redibujo queda pendiente.

Sección 2 Tabla de Transiciones

Convención de acciones: C significa consumir el carácter; K significa conservarlo para que el siguiente reconocimiento lo procese. EOF es una condición de fin, no un carácter consumible. Las tablas representan funciones con anticipación de entrada y sus salidas, de acuerdo con el recorrido manual implementado.

Estado	Entrada	Siguiente o salida	Acción
Q0	Blanco según char.IsWhiteSpace	Q0	C; Avanzar actualiza línea/columna.

Estado	Entrada	Siguiente o salida	Acción
Q0	L o G	QI	C; guardar índice, línea y columna de inicio.
QI	L, D o G	QI	C.
QI	Otro o EOF	A	K si hay carácter; clasificar lexema completo con EsReservada; si no, registrar símbolo.
Q0	D	QN	C.
QN	D	QN	C.
QN	Punto .	QP	C; primer punto, todavía no es decimal válido.
QN	L o G	QNE	C mientras la entrada pertenezca a R; motivo: inicio numérico.
QN	Separador u EOF	A entero	K si hay carácter.
QP	D	QD	C; ya existe al menos un dígito fraccionario.
QP	No D o EOF	QNE / ERR	Consumir únicamente la continuación R si existe; motivo: falta dígito tras el punto.
QD	D	QD	C.
QD	Punto, L o G	QNE	C mientras pertenezca a R; motivo: decimal mal formado.
QD	Separador u EOF	A decimal	K si hay carácter.
QNE	R = L, D, G o punto	QNE	C; incluir toda la secuencia inválida en el error.
QNE	Fuera de R o EOF	ERR	K si hay carácter; registrar error desde la ubicación inicial.

Cadenas: en las filas siguientes, la marca b comienza en falso. No forma parte del texto; registra si apareció algún escape inválido y puede representarse mediante dos variantes finitas de QS/QSE.

Estado	Entrada	Siguiente o salida	Acción
Q0	Comilla doble	QS	C; comienza el lexema.
QS	C_s	QS	C.
QS	Barra invertida	QSE	C.
QSE	E	QS	C; conservar b.
QSE	Otro excepto CR/LF/EOF	QS	C; establecer b=true.
QS	Comilla doble	A o ERR	C; si b=false emitir TK_CADENA; si b=true informar escape inválido.

Estado	Entrada	Siguiente o salida	Acción
QS o QSE	CR, LF o EOF	ERR	K si es salto; informar cadena sin cerrar.

Caracteres literales: k se limita conceptualmente a 0, 1 o más de 1; cada unidad normal o cada par de escape suma una unidad. b señala escape inválido. La implementación usa un contador y un booleano; las variantes finitas permiten dibujar el reconocedor sin afirmar que existe un parser.

Estado	Entrada	Siguiente o salida	Acción
Q0	Comilla simple	QC0	C; k=0, b=false.
QCk	C_c	QC(k+1)	C; saturar la categoría conceptual de k en más de 1.
QCk	Barra invertida	QCEk	C.
QCEk	E	QC(k+1)	C; una unidad por escape.
QCEk	Otro excepto CR/LF/EOF	QC(k+1)	C; b=true.
QCk	Comilla simple	A o ERR	C; aceptar solo con k=1 y b=false. Si b=true, informar escape inválido; de lo contrario, cantidad incorrecta.
QCk o QCEk	CR, LF o EOF	ERR	K si es salto; informar carácter sin cerrar.

Comentarios: Q0 anticipa el segundo carácter para priorizar // y /* sobre división. Las transiciones QB/QB* que siguen expresan el mismo cierre que LeerComentarioBloque implementa anticipando */.

Estado	Entrada	Siguiente o salida	Acción
Q0	Par //	QL	Consumir las dos diagonales.
QL	C_l	QL	C.
QL	CR, LF o EOF	A	K si es salto; emitir TK_COMENTARIO_LINEA.
Q0	Par /*	QB	Consumir apertura.
QB	No asterisco	QB	C; CR/LF actualizan posición.
QB	Asterisco	QB*	C.
QB*	Asterisco	QB*	C; conservar la posibilidad de cierre.
QB*	Diagonal /	A	C; primer cierre: emitir TK_COMENTARIO_BLOQUE.
QB*	Otro carácter	QB	C; sigue siendo contenido del bloque.

Estado	Entrada	Siguiente o salida	Acción
QB o QB*	EOF	ERR	Comentario multilínea sin cerrar; el error abarca hasta EOF.

En la fila QB con no asterisco, EOF se trata por la última fila y no como carácter. En QB*, otro carácter excluye asterisco, diagonal y EOF. Los prefijos /* internos son texto y no aumentan una profundidad de anidamiento.

Familia	Pares reconocidos antes de los simples	Símbolos simples
Aritméticos	++ --	+ - * / %
Relacionales	== != <= >=	< >
Lógicos	&& ||	!
Asignación	+= -= *= /= %=	=
Delimitadores	No hay pares.	( ) { } [ ] ; , . :

QO consume dos caracteres solo cuando el par es válido. Si no lo es, consume únicamente el símbolo simple y conserva el siguiente. Una barra aislada o seguida de = se trata como operador una vez descartados comentarios. & y | aislados no son operadores admitidos: generan error de carácter no permitido. Cualquier otra unidad no definida consume una unidad y genera un error antes de volver a Q0.

Sección 3 Arquitectura UML

Clase y archivo	Responsabilidad real	Relaciones y operaciones relevantes
AnalizadorLexico / Core	Cursor, posiciones, listas, reconocedores y recuperación manual.	Usa AutomataAFD y TablaDeSimbolos; crea Token y ErrorLexico. Analizar(string), AnalizarArchivo(string), ObtenerTokens(), ObtenerErrores(), ObtenerSimbolos(), TotalLineas.
AutomataAFD / Core	Predicados compartidos, conjunto de reservadas y escapes.	EsDigito(char), EsInicioIdentificador(char), EsParteIdentificador(char), EsReservada(string), EsEscapeValido(char). No es la clase que recorre el archivo.
Token y ErrorLexico / Core	Modelos de salida con lectura pública y asignación privada.	Token: Lexema, TipoToken, Linea, Columna. ErrorLexico: CaracterInvalido, Linea, Columna, Descripcion.
Simbolo y TablaDeSimbolos / Core/TablaDeSimbolos.cs	Modelo de identificador y registro único ordenado.	Diccionario ordinal y lista de orden. AgregarSimbolo y ObtenerSimbolos. No duplicar Simbolo en otro archivo.
FrmPrincipal / UI	Carga, editor, numeración, eventos, estado, tablas y estadísticas.	Clase parcial con FrmPrincipal.Designer.cs y FrmPrincipal.resx; usa AnalizadorLexico, ResultadoAnalisis y Exportador. Sustituye el nombre conceptual FormPrincipal.

Clase y archivo	Responsabilidad real	Relaciones y operaciones relevantes
ResultadoAnalisis / UI	Instantánea del análisis para presentación y exportación.	Tokens, Errores, Simbolos, TotalLineas, TiempoMs, NombreFuente, TextoAnalizado y FechaHora. Copia las listas de entrada; sus propiedades List siguen siendo mutables.
Exportador / UI	Escritura del informe TXT y escapes reversibles.	ExportarTxt y EscaparCadena/DesescaparCadena. Recibe los objetos del resultado; no reconstruye datos desde las celdas.
Program / raíz	Punto de entrada de Windows Forms.	Main con STAThread y Application.Run(new FrmPrincipal()).

Relaciones para el UML: Program inicia FrmPrincipal; FrmPrincipal usa AnalizadorLexico y crea ResultadoAnalisis; AnalizadorLexico usa AutomataAFD y contiene una TablaDeSimbolos; las listas del núcleo contienen Token y ErrorLexico; TablaDeSimbolos contiene Simbolo; ResultadoAnalisis mantiene listas copiadas de esos modelos; Exportador depende de ResultadoAnalisis y de los modelos para escribir el informe.

Controles reales de FrmPrincipal: rtbEditor, pnlNumeros, dgvTokens, dgvErrores, dgvSimbolos, txtEstadisticas y etiquetas de resumen/estado. Eventos: btnCargar_Click, btnAnalizar_Click, btnExportar_Click y btnLimpiar_Click. El estado de resultados se invalida al editar o si ocurre una excepción durante el análisis.

La lectura estricta pertenece a FrmPrincipal.IntentarLeerArchivo y a su auxiliar privado IntentarDecodificarArchivo. El método del núcleo AnalizarArchivo todavía usa File.ReadAllText; no se le atribuye la misma política estricta de la interfaz. Los cambios de formato del informe y de numeración no modifican la gramática léxica.

Sección 4 Repositorio y Mockup

Conservar el mockup y las capturas históricas como diseño del Entregable 2. En la actualización del Entregable 4 añadir las capturas reales del manual y una nota que explique su evolución. La publicación y el tag/release del E4 siguen pendientes de verificar; no sustituir el enlace histórico por una publicación que no se haya comprobado.

Actualización del Entregable 3

Documento de destino: Entregable3_final.pdf. Conservar intacto el informe original y sus 28 verificaciones. Incorporar los textos siguientes como revisión posterior vinculada al Entregable 4, o en una copia revisada cuya fecha de actualización sea explícita.

Alcance de esta etapa y Arquitectura

Actualización posterior: el núcleo del Entregable 3 se integró con FrmPrincipal, una interfaz Windows Forms con editor, numeración, tablas independientes, estadísticas y exportación TXT. Se mantienen C-extended y el reconocimiento manual. Las clases de presentación ResultadoAnalisis y Exportador pertenecen a UI y no convierten el motor en un analizador sintáctico o semántico.

Decisiones de implementación y Dificultades y soluciones

Se corrigió el consumo parcial de algunas secuencias numéricas inválidas. La versión recibida para revisión podía reportar 12. como error y registrar abc como identificador al leer 12.abc. La revisión consume 12.abc y 1..2abc completos como un único error numérico y conserva el separador y los tokens válidos posteriores. Este cambio completa la intención de recuperación descrita en el informe histórico, sin afirmar que todos esos casos ya habían sido probados entonces.

Se introdujo EsDigito como predicado ASCII compartido y se sustituyeron las llamadas numéricas a char.IsDigit, que aceptaban también otras cifras Unicode. La definición formal de D y la continuación de identificadores ahora coinciden con los dígitos 0-9. Se conservan las letras admitidas por char.IsLetter y el guion bajo.

La interfaz valida todo el texto decodificado para rechazar NUL antes de asignarlo al editor. Se eliminó la numeración final adicional; se agregó InvalidarResultados al manejo de excepciones de análisis; ResultadoAnalisis copia las listas recibidas. Se alineó App.config con .NET Framework 4.8. Estos cambios de entrada, presentación y estado no deben presentarse como nuevas categorías léxicas.

Pruebas y resultados

Resultado histórico que se conserva: 28 verificaciones aprobadas y 0 fallidas en el Entregable 3. La suite actual del proyecto contiene 76 comprobaciones y produjo 76 aprobadas y 0 fallidas. Los cinco conteos históricos de tokens/símbolos/errores se mantienen; la revisión no reemplaza las 28 verificaciones antiguas por 76 como si ambos resultados fueran de la misma fecha.

Archivo del proyecto actual	Tokens	Símbolos	Errores	Estado
prueba1_sencillo.txt	25	3	0	Aprobado
prueba2_medio.txt	62	9	0	Aprobado
prueba3_errores.txt	20	4	4	Aprobado
prueba4_recuperacion.txt	21	4	2	Aprobado
prueba5_categorias.txt	59	7	0	Aprobado
prueba6_interfaz_exportacion.txt original de AAA2/Pruebas	67	11	1	Aprobado; 30 líneas
Prueba 6 nueva de la carpeta documental	98	15	1	Aprobado; 32 líneas

Las dos pruebas 6 tienen contenido distinto y se conservan separadas. La nueva está en Entradas/Prueba6_nueva y coincide con las capturas del manual y la exportación real del usuario. Sus conteos distintos no son una regresión. En prueba3 también existen diferencias de espacios o salto final entre copias: la del proyecto actual tiene 20 líneas y la copia documental histórica 19, conservando 20 tokens, 4 símbolos y 4 errores.

Conjunto de evidencia actual	Resultado	Qué verifica
Suite del proyecto	76/76	Continuidad de los seis archivos del proyecto, carga, exportación, construcción del formulario y sus controles.
Carga estricta	25/25	Codificaciones válidas e inválidas, vacíos, NUL en distintas posiciones y rechazo de UTF-32.
Números y recuperación	28/28	Lexema inválido completo, ubicaciones, separador posterior, enteros/decimales y continuidad histórica.
Dígitos e identificadores	12/12	Predicado ASCII en 65536 valores char y ejemplos de reconocimiento.
Exportación real de la nueva prueba 6	114 registros coincidentes	98 tokens, 1 error y 15 símbolos comparados campo por campo y en orden.

Los conjuntos de comprobaciones tienen coberturas parcialmente relacionadas y no se suman como si fueran archivos o pruebas independientes. El tiempo del análisis depende de la ejecución y se registra en milisegundos enteros; un valor 0 ms no implica ausencia de análisis.

Compilación y ejecución reproducible

El proyecto principal actual y PruebasConsola apuntan ambos a .NET Framework 4.8. Se compiló una copia completa de los archivos guardados en Debug y Release mediante MSBuild de Visual Studio; el resultado fue satisfactorio. App.config y los exe.config generados indican CLR v4.0 y sku .NETFramework,Version=v4.8.

El SDK .NET 10 y el comando dotnet run documentados en el informe del Entregable 3 describen el ejecutor histórico. La suite actual es un proyecto de .NET Framework 4.8 con referencia a la aplicación. Para la versión actual, compilar PruebasConsola.csproj con las herramientas de .NET Framework de Visual Studio y ejecutar su PruebasConsola.exe indicando la carpeta Pruebas como argumento. Conservar la explicación histórica en el original; no presentarla como requisito de instalación del ejecutable final.

Ejemplo actual desde la raíz del proyecto, después de compilar en Debug: PruebasConsola\bin\Debug\PruebasConsola.exe .\Pruebas. La evidencia se escribe en la carpeta de pruebas indicada; conservar por separado cualquier evidencia histórica antes de volver a ejecutarla.

Trazabilidad y alcance pendiente

Se conservaron los archivos originales y las evidencias históricas. El manual de E4 y la matriz de pruebas documentan la versión revisada y distinguen comprobaciones automáticas de confirmaciones manuales del grupo. La ejecución en otro equipo y la publicación de la versión/tag del Entregable 4 siguen pendientes de comprobación. El video del Entregable 4 se aplazó por decisión del grupo; esa decisión no modifica los requisitos de la guía ni el video histórico del Entregable 3.

El flujo excepcional que invalida resultados al fallar un análisis está implementado y compila, pero no se provocó una excepción real del motor durante la revisión. Se conserva ese límite en el registro; no se atribuye una prueba forzada que no se ejecutó.

Reconocimiento de apoyo de IA

Se empleó Antigravity como apoyo para la implementación e integración de Windows Forms, y Codex para revisar el código y los documentos, proponer correcciones, ejecutar comprobaciones y preparar documentación. El grupo aplicó los cambios y realizó las pruebas manuales descritas. Las capturas proceden de la aplicación del grupo en ejecución. Este reconocimiento atiende la norma de la guía sobre citar explícitamente el apoyo de herramientas de IA.